



SCHOOL OF ENGINEERING, APPLIED SCIENCE,
AND TECHNOLOGY

2025-2026

FINAL YEAR PROJECT REPORT

On

**Real-Time Gesture Recognition with
MediaPipe Landmarks and KNN Models
for Interactive Command Control**

Course: BCS410

Submitted by:

Ossama Alhussni ID: 20220001208

Noureddin Albaba ID: 20220001550

Mohammed Aasim ID: 20220002173

In partial fulfillment of the requirements for a Bachelor of
Science in Computer Science

Abstract

Conventional input methods such as touchscreens and keyboards are impractical or unsafe in a range of everyday contexts, particularly in automotive environments where manual interaction with infotainment displays was shown to contribute significantly to driver distraction and collision risk. For users with motor or speech impairments, these barriers further limited access to common computing tasks. This project addressed the need for a lightweight, vision-based, hands-free command interface deployable on standard consumer hardware without cloud dependency. A real-time gesture recognition and command execution system was developed using a standard laptop webcam. Google's MediaPipe Hands solution was used to extract 21 hand landmarks per frame, which were normalized relative to the wrist to produce a scale-invariant feature vector and classified into one of five predefined gesture classes using a K-Nearest Neighbours classifier. A temporal smoothing buffer with majority voting and a configurable hold timer were applied to prevent accidental triggers. Each confirmed gesture was mapped to a distinct executable action through a modular action dispatch layer. A Flask web server was integrated to stream a live annotated webcam feed alongside real-time gesture state, action history, and voice assistant feedback through a browser-accessible dashboard. The KNN classifier achieved 88% overall offline accuracy, with perfect classification of the thumbs-up gesture. All five gesture-to-action mappings were verified through 17 system test cases, all of which passed. The voice assistant successfully transcribed and executed spoken commands in a quiet indoor environment. Dashboard settings were confirmed to update live system behaviour without requiring a restart. The results demonstrated that a reliable, end-to-end gesture command system could be built on CPU-only hardware with real-time performance, providing a practical foundation for hands-free interaction in automotive infotainment and accessibility contexts.

Acknowledgments & Dedication

We would like to thank our supervisor Dr. Najla for her guidance and support throughout the project. Special thanks to the developers of MediaPipe, scikit-learn, and the SpeechRecognition library for providing the open-source tools that made this system possible. The HaGRID dataset, made publicly available by Kapitanov et al., was crucial in training the gesture classifier. This work is dedicated to our families and friends for their continued support and encouragement.

Table of Contents

Abstract	i
Acknowledgments & Dedication	ii
Table of Contents.....	iii
List of Figures.....	v
List of Tables.....	vi
Chapter 1: Introduction	1
1.1 Background & Motivation.....	1
1.2 Problem Formulation	2
1.3 Objectives.....	3
1.4 Report Organization	4
Chapter 2: Literature Review & Comparison	5
2.1 Review of Related Work.....	5
2.2 Comparative Analysis Table.....	8
2.3 Conclusions on Literature	9
Chapter 3: System Analysis & Design	11
3.1 Requirements Gathering	11
3.2 Functional & Non-Functional Requirements	11
3.3 High-Level System Design	14
Chapter 4: Implementation Details.....	17
4.1 Technology Stack.....	17
4.2 Development Phases.....	18
4.3 Core Algorithms	21
4.3.1 Landmark Normalisation.....	21
4.3.2 KNN Inference and Temporal Smoothing	21
4.3.3 Hold Timer and Action Dispatch	22
4.3.4 Gesture-to-Action Mapping.....	23
4.3.5 Voice Assistant Command Recognition	24
4.3.6 Main Inference Loop.....	26
4.4 Implementation Challenges	27
Chapter 5: Results, Testing, & Evaluation	28
5.1 Test Methodology.....	28
5.2 Comprehensive Test Results	29

5.2.1 Offline Classification Results	29
5.2.2 System Test Results.....	31
5.3 Analysis of Strengths & Limitations.....	36
5.4 Impact Analysis	37
5.4.1 Security and Privacy	37
5.4.2 Ethics	37
5.4.3 Societal Impact	38
Chapter 6: Conclusion & Future Work.....	39
6.1 Project Conclusion	39
6.2 Recommendations for Future Improvements	40
References	41
Appendix A: Plagiarism Report.....	43
Appendix B: Team Contribution Statement	44

List of Figures

<i>Figure 1.1 Chosen predefined gestures for the system.....</i>	<i>3</i>
<i>Figure 3.1 High-Level System Architecture.....</i>	<i>14</i>
<i>Figure 5.1 Confusion matrix.....</i>	<i>31</i>
<i>Figure 5.2 Verification of gesture recognition.....</i>	<i>34</i>
<i>Figure 5.3 Dashboard screenshot.....</i>	<i>35</i>
<i>Figure 5.4 Configurable hold time and SOS number.....</i>	<i>35</i>

List of Tables

<i>Table 2.1 Comparative Analysis</i>	8
<i>Table 3.1 System Requirements</i>	11
<i>Table 4.1 Gesture-to-Action Mapping, Normal Mode</i>	23
<i>Table 4.2 Gesture-to-Action Mapping, PowerPoint Mode</i>	24
<i>Table 4.3 Voice Assistant Recognised Commands</i>	25
<i>Table 5.1 KNN Classifier Performance per Gesture Class (k=5, Euclidean Distance)</i>	29
<i>Table 5.2 System Test Results</i>	31
<i>Table B.1 Individual Team Contributions</i>	44

Chapter 1: Introduction

1.1 Background & Motivation

Human-computer interaction has evolved steadily from physical keyboards and mice toward more natural, contact-free input modalities. Computer vision has emerged as a practical foundation for gesture-based interfaces, driven by the wide availability of webcams in consumer laptops, smartphones, and vehicles. The proliferation of these devices means that a vision-based interaction system requires no specialised hardware, making deployment at scale both technically feasible and economically viable today in a way that was not possible a decade ago [1]. Within this broader shift, hand gesture recognition has attracted sustained research interest across augmented and virtual reality, gaming, smart home control, and sign language interpretation [1]. Google's MediaPipe Hands solution marked a significant milestone by enabling real-time extraction of 21 three-dimensional hand landmarks on standard CPU hardware without GPU acceleration [2]. This development lowered the barrier for gesture-based system control from research-grade hardware to any consumer laptop, making the practical deployment of such systems timely and relevant. The automotive industry illustrates the social and safety significance of this technology most sharply. BMW's gesture-controlled infotainment interface, for example, allows drivers to execute several commands through simple hand movements, demonstrating practical deployment of gesture recognition in safety-critical environments [3]. Studies report that manual interaction with touch-based infotainment screens contributes to one in every three road accidents, highlighting the need for non-contact, eyes-free alternatives [4]. Gesture interfaces address this by enabling command input without requiring the driver to look away from the road. Accessibility provides an equally compelling motivation. Users with speech impairments cannot rely on voice assistants, and users with limited motor function may struggle with keyboards and touchscreens. Gesture interfaces that operate through a standard webcam and require only simple hand movements offer an inclusive, low-cost alternative input channel that existing solutions do not adequately address.

1.2 Problem Formulation

Three distinct but related problems motivate this project, each quantifiable and unresolved by existing systems. First, in automotive environments, manual interaction with touchscreen infotainment systems introduces significant driver distraction by requiring prolonged visual and cognitive engagement with secondary tasks. Research conducted by the AAA Foundation for Traffic Safety reported that drivers using in-vehicle infotainment interfaces for tasks such as navigation entry and text messaging experienced distraction periods exceeding 40 seconds [5]. This visual distraction has been identified as a contributing factor in a significant proportion of road collisions. Current infotainment interfaces do not provide a reliable, standardised gesture input layer that allows drivers to execute common commands, such as navigating, muting audio, or initiating emergency contact, without diverting their attention. Second, existing gesture recognition research offers no practical middle ground between two extremes: deep learning models such as VGGNet achieve up to 97% classification accuracy but require GPU hardware, datasets of tens of thousands of labelled images, and substantial training time, making them unsuitable for low-cost deployment [6]. At the other extreme, rule-based and threshold systems are fast but brittle, degrading significantly under variations in lighting, background clutter, and hand orientation. A lightweight classifier operating on normalised hand landmarks, trained on a small per-class dataset, and running on a CPU with under 200 ms end-to-end latency, represents an unoccupied and practically valuable position in this design space. Third, no existing prototype integrates gesture recognition with a complete action execution layer, a voice assistant fallback, an emergency notification feature, and a configurable real-time dashboard within a single deployable system. Each of these components exists independently in the literature, but their integration into a coherent, end-to-end command interface running entirely on-device has not been demonstrated. This project directly addresses all three problems by developing and evaluating such a system on a standard consumer laptop, without cloud dependency, GPU hardware, or large-scale training data.

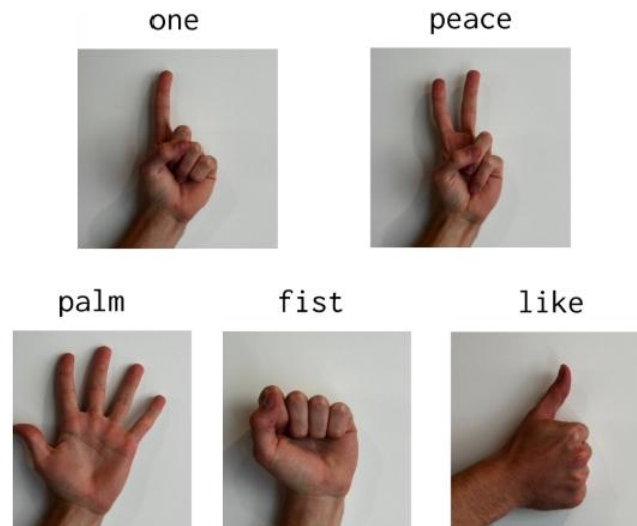


Figure 1.1: Chosen predefined gestures for the system

1.3 Objectives

- Develop a real-time gesture recognition pipeline using MediaPipe landmark extraction and a KNN classifier achieving at least 85% offline classification accuracy across five gesture classes.
- Ensure real-time performance with end-to-end latency below 200 ms and sustained frame processing above 10 FPS on CPU-only hardware.
- Map each of the five recognised gestures to a distinct, executable system action including navigation, audio control, application launch, emergency notification, and voice assistant activation.
- Implement a voice-activated assistant triggered by the thumbs-up gesture, capable of recognising and executing spoken commands hands-free using Google Speech-to-Text.
- Implement an emergency SOS feature that opens WhatsApp Web with a pre-configured contact and pre-filled message when the open-palm gesture is held.
- Build a real-time Flask web dashboard that streams the annotated webcam feed, displays live gesture state and action history, shows voice assistant feedback, and provides a settings panel for runtime configuration.

1.4 Report Organization

Chapter 2 presents a review of related work in gesture recognition and a comparative analysis of existing solutions against this project. Chapter 3 covers system analysis and design, including a complete requirements specification and high-level architecture. Chapter 4 details the implementation including the technology stack, development phases, and core algorithms. Chapter 5 presents testing methodology and comprehensive evaluation results. Chapter 6 concludes the report and provides recommendations for future development.

Chapter 2: Literature Review & Comparison

2.1 Review of Related Work

Ben et al. surveyed gesture recognition approaches and drew a clear distinction between glove-based and vision-based systems [7]. Glove-based systems embed joint-angle sensors directly on the hand and achieve high precision under controlled conditions. However, their primary limitations are cost, bulk, and the requirement for users to wear specialised equipment, which is impractical in automotive or accessibility contexts. Vision-based systems, by contrast, require only a camera. Their limitation is sensitivity to lighting variation and background clutter, which can degrade detection reliability in uncontrolled environments. This project adopts a vision-based approach to maximise accessibility, accepting the robustness trade-off and addressing it through landmark normalisation. Zhang et al. introduced MediaPipe Hands, a two-stage pipeline that first detects the hand bounding box using a lightweight palm detector and then estimates 21 three-dimensional landmark coordinates within that region at real-time speeds on CPU hardware [2]. Its key strength is efficiency: by tracking landmarks between frames after initial detection, the computational cost per frame is substantially reduced. A notable limitation is that the system is designed for one to two hands and degrades under severe occlusion or when the hand exits and re-enters the frame, requiring re-detection. Despite this, MediaPipe Hands has become the de facto standard for landmark-based gesture prototypes and forms the detection backbone of this project. Moysiadis et al. developed an integrated real-time gesture recognition framework for controlling an unmanned ground vehicle in agricultural settings, combining MediaPipe landmark extraction with multiple classical machine learning classifiers evaluated across varied environmental conditions [8]. The key strength of this work is its ecological validity: testing under real outdoor conditions with varying lighting and background provides evidence that landmark-based KNN approaches generalise beyond controlled lab settings. Its limitation is domain specificity; the gesture set and action mapping were designed exclusively for agricultural robotics and do not transfer to desktop or automotive command interfaces. This study nonetheless directly validates the core technical choices of MediaPipe and KNN adopted in this project. Bhushan et al. conducted a systematic experimental comparison of KNN, Support Vector Machines, Naive Bayes, and Decision Tree classifiers applied to hand gesture recognition [9]. KNN consistently outperformed the alternatives across multiple

gesture sets, which the authors attributed to its exploitation of local neighbourhood structure in the landmark feature space, a property well-suited to the continuous, geometrically structured nature of hand pose data. The study's strength is its breadth of systematic comparison across classifiers. Its limitation is that evaluation was performed offline on pre-extracted features without a real-time inference loop or latency analysis, leaving open the question of practical deployment performance. Duwairi and Halloush applied a VGGNet deep convolutional network to Arabic sign language alphabet recognition, achieving 97% accuracy [6]. The strength of this work is its high classification ceiling, demonstrating what is achievable when large, labelled datasets and GPU hardware are available. Its limitations are substantial: the model requires a GPU for both training and inference, a dataset of thousands of labelled images per class, and significant training time, all of which make it unsuitable for deployment on standard consumer hardware or embedded systems. Savas and Erguzen extended deep learning gesture recognition using transfer learning and ensemble methods, reporting strong accuracy with improved generalisation over single-model approaches [10]. The study's strength is its demonstration that transfer learning reduces the data requirement compared to training from scratch. However, the approach still requires GPU infrastructure and pre-trained model weights, representing a practical deployment barrier. The authors noted that ensemble overhead increases inference latency, which conflicts with real-time requirements below 200 ms. Bao et al. evaluated a compact convolutional network for gesture recognition without explicit hand localisation [11]. In simple, uncluttered environments the model achieved 97% accuracy. However, accuracy fell to approximately 85% under complex backgrounds, a 12-percentage-point degradation that quantifies the background sensitivity limitation of pixel-based approaches. This result directly motivates the landmark-based approach in this project: MediaPipe isolates the hand through its palm detector and landmark regression, effectively decoupling classification from background variation and avoiding this degradation. Kapitanov et al. released HaGRID, a large-scale dataset comprising over 552,000 images across 18 command-oriented gesture classes, collected from over 34,000 subjects in diverse real-world settings with varied lighting, background, and subject demographics [12]. The dataset's strength is its scale and ecological diversity, which directly addresses limitations of earlier small-scale, lab-controlled gesture datasets. Its limitation for this project is that only five of the eighteen classes were relevant, and the raw images required MediaPipe preprocessing to extract landmark vectors before they could be used for KNN training. This project uses HaGRID images for the five selected gesture classes.

Zhang et al. compared touchscreen, speech, and gesture-based interaction modalities for in-vehicle infotainment systems using a simulated driving environment [13]. Their findings showed that touchscreen interaction produced the highest visual demand and the greatest deterioration in driving performance, while gesture-based interaction resulted in fewer and shorter off-road glances and reduced distraction effects. A major strength of the study is its direct evaluation of driver visual behaviour and vehicle control metrics under controlled experimental conditions. However, the work focused on interaction modality effects rather than the design or robustness of the underlying gesture recognition system. Remiro et al. investigated the effect of localisation and zoom normalisation on MediaPipe-based hand pose recognition accuracy using ASL datasets [14]. Their experiments showed that wrist-relative translation and scale normalisation substantially improved classification accuracy, particularly under high variability in hand position and size. The study provides empirical support for the use of landmark normalisation in gesture recognition systems. However, evaluation was conducted on pre-collected datasets rather than within a deployed real-time application, and the work did not include explicit latency measurements for end-to-end inference performance. Alksasbeh et al. evaluated KNN specifically for hand gesture recognition applied to video annotation tasks, reporting that KNN achieved strong accuracy with low computational overhead compared to more complex models [15]. Their study reinforces the suitability of KNN for small, well-defined gesture vocabularies. The limitation of their work is that the gestures evaluated were annotation-specific and not transferable to general command-control scenarios, and no real-time deployment or action execution was implemented. Parada-Loira et al. [4] present an in-vehicle vision-based gesture control system that uses a ceiling-mounted camera and classical computer vision techniques to recognize predefined static and dynamic hand gestures for infotainment interaction, enabling touchless control of car functions in a constrained driving environment. A key limitation of this system is its reliance on hand-crafted rules and manually designed decision logic, which reduces adaptability and robustness when exposed to unconstrained or highly variable real-world driving conditions.

2.2 Comparative Analysis Table

Table 2.1 Comparative Analysis

Criterion	Moysiadis et al. [8]	Bhushan et al. [9]	Duwairi & Halloush [6]	Parada-Loira et al. [4]	This Project
Approach	MediaPipe, KNN/SVM	Multi-classifier ML	VGGNet deep CNN	Rule-based vision	MediaPipe, KNN, action layer, Flask dashboard
Dataset / Platform	Custom outdoor (ZED2 camera)	Benchmark gesture sets	Arabic sign language images	Custom in-vehicle camera	HaGRID (5 classes, landmark vectors)
Year	2022	2022	2022	2014	2026
Classifier	KNN, SVM	KNN, SVM, NB, RF	VGGNet	Rule-based (EBM, MoG, MSER)	KNN (k=5, Euclidean distance)
Accuracy	Around 85%	Around 90% (KNN achieving 81%)	97%	Not reported	88% offline; stable in live operation
Real-time	Yes	N/A	No	Yes	Yes (14+ FPS, <200 ms latency)

GPU required	No	No	Yes	No	No (CPU only)
Action execution	Robot vehicle control	None	None	Basic infotainment	5 system actions
Web dashboard	No	No	No	No	Yes (Flask, live MJPEG stream, settings panel)
On-device privacy	Yes	Yes	Yes	Yes	Yes (fully local, no cloud)

2.3 Conclusions on Literature

The reviewed works reveal three persistent gaps that none of the existing solutions individually or collectively address. The first gap is the absence of a complete, integrated action execution layer. Of the ten works reviewed, only Parada-Loira et al. [4] connect gesture recognition to actual system commands. However, the system relies on rule-based gesture detection and hand-crafted decision logic rather than learning-based models, which may limit adaptability to more unconstrained real-world driving environments. The remaining works treat classification as the end goal, with no implementation of what should happen when a gesture is recognised. This project addresses this gap by implementing a five-gesture action dispatch layer with a dedicated action class per command, a configurable hold timer, and a gesture lock mechanism. The second gap is the lack of multi-modal command input. None of the reviewed works combine gesture recognition with a voice-activated fallback. In real-world deployment, particularly in automotive contexts, there are scenarios where a gesture may be ambiguous or unavailable. This project addresses this by integrating a voice assistant triggered by a dedicated gesture, allowing the user to switch between input

modalities without leaving the system. The third gap is the absence of a real-time feedback and configuration interface. None of the reviewed works provide a live dashboard that consolidates gesture state, action history, voice feedback, and runtime settings in a single accessible view. This project addresses this by implementing a Flask dashboard with a live MJPEG stream, an information display, and a settings panel allowing the SOS contact number and hold time to be changed without restarting the application. Taken together, these gaps justify this project's contribution: a deployable, end-to-end gesture command system that goes beyond recognition to deliver practical, configurable, multimodal human-computer interaction on standard consumer hardware.

Chapter 3: System Analysis & Design

3.1 Requirements Gathering

Requirements were gathered through three methods. First, a review of HCI literature and automotive safety studies identified the need for low-latency, eyes-free input with clear visual feedback and configurable parameters [4], [16]. Second, the System Usability Scale framework informed the design of the feedback interface, emphasising the importance of visible interaction cues, clear gesture feedback, and action confirmation mechanisms [17]. Third, informal discussions with potential users highlighted the value of a dashboard that consolidates all system information in a single view, and the importance of a configurable hold time and emergency contact to support diverse users.

3.2 Functional & Non-Functional Requirements

Table 3.1 System Requirements

ID	Requirement Description	Priority	Status
FR-1	The system shall access the device webcam and display the live annotated video feed.	High	Met
FR-2	The system shall detect one hand per frame and extract 21 normalized landmark coordinates in real time.	High	Met
FR-3	The system shall classify each frame into one of five predefined gesture classes or as 'no gesture' using a KNN model.	High	Met
FR-4	The system shall apply majority-vote smoothing over a configurable frame buffer before confirming a gesture.	High	Met

FR-5	The system requires a gesture to be held continuously for a configurable hold time before triggering its associated action.	High	Met
FR-6	The system shall map each confirmed gesture to a distinct executable system action as defined in Table 4.1.	High	Met
FR-7	The system shall include a post-action cooldown preventing the same gesture from re-triggering within a defined interval.	High	Met
FR-8	In PowerPoint mode, index-point shall advance to the next slide and thumbs-up shall return to the previous slide.	High	Met
FR-9	The system shall serve a real-time web dashboard on localhost displaying the live webcam stream, current gesture, confidence score, last action, and voice assistant feedback.	High	Met
FR-10	The dashboard settings panel shall allow runtime updates to the SOS contact number and hold time without restarting the application.	Medium	Met
FR-11	The system shall display an always-visible gesture legend on the dashboard reflecting if the current mode is normal or PowerPoint.	Medium	Met
NFR-1	The system shall process webcam frames at a minimum of 10 FPS with end-to-end latency below 200 ms.	High	Met

NFR-2	The KNN classifier shall achieve a minimum of 85% accuracy on held-out test data.	High	Met (88%)
NFR-3	All video processing shall occur locally on the device; no frames shall be transmitted or stored externally.	High	Met
NFR-4	The interface shall always display the recognized gesture, confidence, last action, and gesture map without requiring user navigation.	High	Met
NFR-5	The classifier and action registry shall support addition of new gesture classes by retraining with extended data, with no architectural changes.	Medium	Met

3.3 High-Level System Design

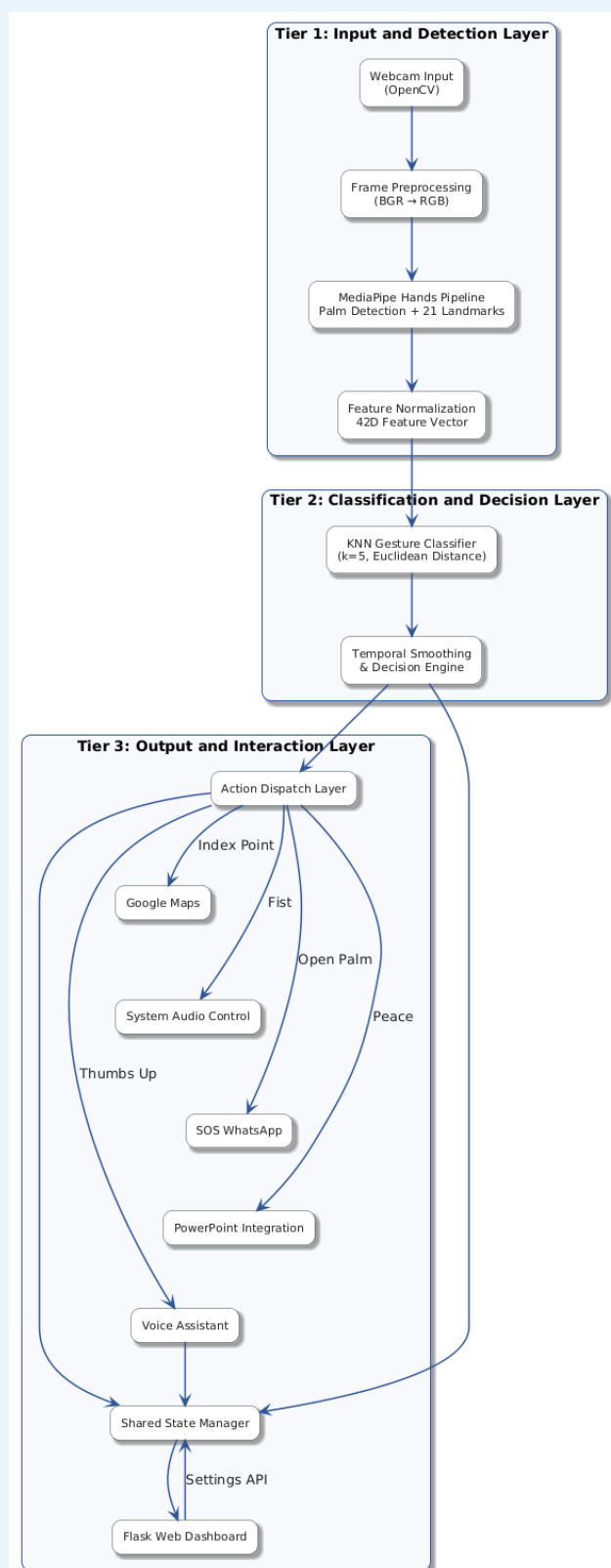


Figure 3.1 High-Level System Architecture

Webcam Input Module

The Webcam Input module captures live video frames from the system webcam using OpenCV. These frames act as the raw input stream for the gesture recognition pipeline and are processed in real time.

Frame Preprocessing Module

The Frame Preprocessing module converts frames from BGR to RGB colour space, which is required by MediaPipe. It also prepares frames for efficient hand landmark detection.

MediaPipe Hands Pipeline

The MediaPipe Hands Pipeline detects the hand region and extracts 21 three-dimensional hand landmarks. These landmarks provide spatial key points used for gesture recognition.

Feature Normalization Module

The Feature Normalization module converts landmark coordinates into a scale-invariant representation. The wrist is used as the origin and values are normalized to produce a 42-dimensional feature vector.

KNN Gesture Classifier

The KNN Gesture Classifier predicts the gesture class using a $k=5$ Euclidean-distance model. It also outputs a confidence score for each prediction.

Temporal Smoothing and Decision Engine

This module stabilizes predictions using a sliding window and majority voting. A gesture is only accepted if it meets confidence and hold-time thresholds.

Action Dispatch Layer

The Action Dispatch Layer maps confirmed gestures to system actions such as opening Google Maps, toggling mute, launching PowerPoint, sending SOS messages, or activating the voice assistant.

Shared State Manager

The Shared State Manager is a thread-safe data store shared across modules. It stores live system data such as the current gesture, confidence, last action, and video frame for real-time access.

Flask Web Dashboard

The Flask Web Dashboard provides a browser interface on localhost:5050. It streams the webcam feed and displays live system updates from the shared state.

Voice Assistant Module

The Voice Assistant runs in a background thread and is triggered by the thumbs-up gesture. It captures speech, processes commands, and returns voice feedback.

Google Maps Service

Triggered by the index_point gesture, this module opens Google Maps with Dubai mall as the destination in the default browser.

System Audio Control

This module toggles system mute/unmute when the fist gesture is detected.

SOS WhatsApp Module

Triggered by the open-palm gesture, it opens a pre-filled WhatsApp SOS message in the browser.

PowerPoint Integration Module

This module enables presentation mode and maps gestures to slide navigation controls.

Data Flow Between Modules

The system starts with webcam frame capture, followed by preprocessing and MediaPipe landmark extraction. The landmarks are normalized and classified using KNN. Temporal smoothing ensures stable predictions before triggering the Action Dispatch Layer. The dispatch layer executes the corresponding system action, while all runtime information is simultaneously stored in the Shared State Manager for the Flask dashboard and other modules to access in real time.

Chapter 4: Implementation Details

4.1 Technology Stack

The system is implemented entirely in Python 3.12, chosen for its extensive library ecosystem, rapid prototyping capability, and the availability of all required dependencies as stable packages. The following libraries and frameworks were selected and justified below.

OpenCV: Provides real-time webcam capture, frame-to-frame annotation, landmark overlay, text, progress bars, and JPEG encoding for the MJPEG video stream served by Flask. Chosen for its Python bindings and GPU-free operation.

MediaPipe: Google's hand landmark detection pipeline. Extracts 21 three-dimensional keypoints at real-time speeds on CPU. Chosen over alternatives due to its robustness to partial obstruction, built-in hand tracking between frames, and its widespread validation in gesture recognition literature.

scikit-learn: Provides the `KNeighborsClassifier` and `LabelEncoder` used in training and inference, and `classification_report` for offline evaluation. Chosen for its well-tested, efficient KNN implementation with probability estimation support.

joblib: Serialises and loads the trained KNN model and label encoder as a single `.pkl` file, enabling offline training and live inference without retraining at runtime.

Flask: Lightweight web framework used to serve the real-time dashboard. Flask runs in a background daemon thread alongside the gesture loop, serving the MJPEG stream via a generator function and the shared state via a JSON REST endpoint. Chosen for its minimal footprint and ease of integration with Python threading.

SpeechRecognition and PyAudio: `SpeechRecognition` provides a high-level interface to Google's free Speech-to-Text API, capturing audio from the microphone and returning a transcribed string. `PyAudio` provides the underlying audio stream. Chosen for their ease of use and zero-cost access to high-accuracy speech transcription without an API key.

pyttsx3: Offline text-to-speech engine that provides audible confirmation of recognised voice commands. Runs fully locally with no network dependency, initialised per-call to ensure thread safety.

NumPy: Used for landmark array manipulation, Euclidean distance computation, and feature vector normalization.

PyAutoGUI: Simulates keyboard media key presses for voice assistant commands that do not have dedicated action classes.

4.2 Development Phases

Development followed an iterative agile methodology with weekly iterations, where each iteration delivered a testable increment of the system.

Phase 1 - Planning: The planning phase defined the overall scope, objectives, and system requirements of the project. Five gesture classes were selected from the HaGRID dataset, specifically `open_palm`, `fist`, `thumbs_up`, `index_point`, and `peace`. The main system goals were identified as real-time gesture recognition, gesture-based action execution, and a web-based dashboard interface. Key milestones were established to guide development, including dataset preprocessing and model training, real-time inference pipeline development, action dispatch system implementation, and Flask dashboard creation. An iterative agile approach was selected to ensure that each component could be developed and tested incrementally across weekly development cycles.

Phase 2 - Design: The design phase focused on defining the system architecture and data flow between modules. The system was designed as a layered pipeline beginning with webcam input captured using OpenCV, followed by frame preprocessing for compatibility with MediaPipe. The MediaPipe Hands pipeline was responsible for detecting hand landmarks, which were then normalised into 42-dimensional feature vectors. These feature vectors formed the input to a KNN classifier. The architecture also included an ActionManager registry responsible for mapping gesture labels to executable system actions, as well as a Flask-based web dashboard providing a real-time interface. A `shared_state` module was designed as a thread-safe communication bridge between the gesture recognition loop and the Flask server, ensuring synchronisation of video frames and system state across concurrent processes.

Phase 3 - Implementation: Implementation was carried out in modular stages aligned with the agile iterations. In the data collection and preprocessing stage, MediaPipe was applied to each image in the selected HaGRID dataset classes to extract landmark coordinates. These coordinates were normalised into vectors, which were stored in class-specific CSV files and then merged into a single `gestures_all.csv` training dataset. In the classifier training stage, a KNN model with k equal to 5 and Euclidean distance was trained using scikit-learn, and both the trained model and label encoder were serialised using joblib. Offline evaluation was used to ensure high accuracy.

The live inference system was implemented in `live_predict.py` and integrated OpenCV webcam capture, MediaPipe landmark detection, feature normalisation, KNN inference, and a smoothing buffer using majority voting to stabilise predictions. A hold timer and post-action cooldown mechanism were introduced to prevent accidental or repeated gesture triggers. The action dispatch layer was implemented using an `ActionManager` class that mapped gesture labels to executable action classes. Individual actions such as `OpenGoogleMapsHome`, `MuteToggle`, and `OpenPowerPoint` were implemented in separate modules under an `actions` directory, with each action class implementing a `trigger` method, enabling a uniform execution interface that executes gesture-specific functionality without requiring conditional handling for individual actions.

Gesture mappings included the `index_point` gesture, which opens Google Maps navigation to Dubai Mall in the default web browser using `webbrowser.open()`, and the `fist` gesture, which triggers the `MuteToggle` action to toggle system audio mute state. The `peace` gesture was extended to activate a `ppt_mode` flag, enabling dynamic switching of gesture mappings for presentation control. The voice assistant was implemented in the `VoiceAssistant` class within `actions/voice_assistant.py`. Triggered by the thumbs up gesture, it was designed to accept a shared `ActionManager` instance so that changes such as PPT mode updates could propagate correctly. When triggered, it spawns a daemon thread, performs microphone input capture that listens for up to six seconds, and transcribes speech using Google Speech-to-Text before dispatching the corresponding action. Feedback from voice interactions is stored in shared state for display on the dashboard. The open palm gesture activates the `SOS WhatsApp` module, which reads the emergency phone number from shared state at runtime, allowing updates from the dashboard without restarting the system. It constructs a `wa.me` URL with a URL-encoded SOS message and opens it in the default web browser.

The Flask web dashboard was implemented alongside a `shared_state` module that acts as a bridge between the inference loop and the web interface. The gesture loop continuously writes the latest annotated frame, protected by a threading lock, along with a state dictionary containing `gesture`, `confidence`, `last_action`, `voice_heard`, `voice_action`, `sos_phone`, and `hold_time` after each frame. The Flask server exposes a `video_feed` route for MJPEG streaming, an `api/state` route for returning the current system state as JSON, and an `api/settings` route that validates and updates configuration values such as `sos_phone` and `hold_time`. The Flask application runs in a background daemon thread started from `live_predict.py`. The dashboard frontend was implemented as a single-page application using HTML, CSS, and JavaScript. A JavaScript polling loop updates the interface every 500 milliseconds using data from the API.

Phase 4 - Testing: Testing was conducted across unit, integration, and system levels to ensure correctness and stability of the system. Unit testing was performed on individual components such as MediaPipe feature extraction, KNN prediction, and action trigger functions to verify correctness in isolation. Integration testing focused on ensuring proper communication between the webcam input, MediaPipe pipeline, classifier, ActionManager, `shared_state` module, and Flask backend. System testing evaluated the full end-to-end pipeline from live webcam input through gesture recognition to action execution and dashboard updates. Real-time performance was assessed to ensure stable recognition with acceptable latency under continuous webcam input.

Phase 5 - Deployment: The system was deployed as a local real-time application designed for demonstration environments rather than cloud hosting due to its reliance on webcam input and real-time processing. The Flask server runs as a background daemon thread initiated from `live_predict.py`, allowing the full system to be launched from a single-entry point script that starts the inference loop, API server, and shared state manager simultaneously. Version control was managed using Git with iterative local commits throughout development, and no external CI or CD pipeline was used. The system is intended to run on a local machine such as a laptop where both webcam access and browser-based dashboard viewing are available, enabling real-time interaction and demonstration of all system features.

4.3 Core Algorithms

4.3.1 Landmark Normalisation

Raw MediaPipe output provides 21 landmark points in normalised image coordinates where x and y each fall in the range $[0, 1]$. These raw coordinates are unsuitable for direct classification because they encode absolute hand position within the frame and vary with hand-to-camera distance and individual hand size. To produce a representation invariant to these factors, the following normalisation is applied before every inference call. Wrist-relative translation removes position dependence. Maximum-distance scaling removes scale dependence without distorting finger-length ratios, which are the primary discriminative features for static gesture classification. This approach was validated by Remiro et al., who demonstrated significant accuracy improvement over unnormalized landmarks under varying distances and orientations [14].

```
FUNCTION normalise(landmarks):
    origin = landmarks[0] // wrist landmark
    FOR each landmark:
        landmark = landmark - origin
    distances = Euclidean distance of each landmark
    max_dist = MAX(distances)
    IF max_dist < 1e-6:
        max_dist = 1.0 // avoid divide-by-zero
    FOR each landmark:
        landmark = landmark / max_dist
    RETURN flattened landmark vector
```

Complexity: Time $O(21) = O(1)$ per frame (fixed landmark count). Space $O(42) = O(1)$ per frame.

4.3.2 KNN Inference and Temporal Smoothing

The normalised 42-dimensional vector is passed to the trained KNN model at every frame. scikit-learn returns a probability distribution over the five gesture classes; the argmax is taken as the frame-level prediction.

Using probability estimates rather than hard predictions enables a per-frame confidence threshold (0.50) that filters unreliable frames before they enter the smoothing buffer. A fixed length deque of size eight was chosen as the smoothing window to balance responsiveness against stability: shorter windows react faster but admit more noise, while longer windows introduce perceptible lag. The majority vote count threshold of four out of eight (50%) ensures that a gesture must genuinely dominate recent history before it is considered stable.

```

FUNCTION infer_and_smooth(vector, buffer):
    probs = knn_model.predict_proba([vector])[0]
    best_idx = ARGMAX(probs)
    confidence = probs[best_idx]
    raw_label = label_encoder.inverse_transform([best_idx])[0]
    // Only keep confident predictions
    IF confidence >= 0.50:
        APPEND raw_label TO buffer
    // Ignore uncertain frames
    IF buffer IS EMPTY:
        RETURN "No hand", 0.0, 0
    vote_counts = COUNT_OCCURRENCES(buffer)
    smoothed_label = LABEL WITH HIGHEST COUNT
    vote_count = vote_counts[smoothed_label]
    RETURN smoothed_label, confidence, vote_count

```

Complexity: Time complexity: $O(n \cdot d)$ per frame for brute-force KNN, where n is the number of training samples and $d = 42$. Since d is fixed by the landmark representation, this simplifies to $O(n)$, linear in training set size. Space complexity: $O(n \cdot d) = O(n)$ to hold all training samples in memory, as KNN stores the entire training set rather than learning an explicit model. Both are effectively constant at a fixed deployment since n does not change at runtime.

4.3.3 Hold Timer and Action Dispatch

Once a smoothed gesture is determined, a hold timer tracks how long it has remained stable. A gesture is eligible for dispatch only when it has been held continuously for at least `HOLD_TIME` seconds (configurable, default 3 seconds), its vote count exceeds four out of eight, and its confidence exceeds 0.80. A `locked_gesture` variable prevents the same gesture from re-triggering within the same hold session. A separate `ACTION_COOLDOWN` of two seconds further limits repeat triggers.

The hold timer eliminates false triggers from brief coincidental poses during normal hand movement. The action lock prevents repeated firing while a user holds a pose. The dual confidence-and-count threshold provides two independent guards against misclassification: a single high-confidence misprediction cannot trigger an action without also dominating the vote buffer.

```

FUNCTION dispatch(vote, vote_count, confidence, state):
    IF vote != state.current_gesture:
        state.current_gesture = vote
        state.gesture_start_time = current_time()
        RETURN
    IF confidence < 0.80 OR vote_count < 4:
        state.current_gesture = None

```

```

    RETURN
    held_time = current_time() - state.gesture_start_time
    IF held_time < HOLD_TIME:
        RETURN
    IF vote == state.locked_gesture:
        RETURN
    IF current_time() - state.last_action_time < ACTION_COOLDOWN:
        RETURN
    // Normal mode
    IF state.ppt_mode == FALSE:
        IF vote == "thumbs_up":
            voice_assistant.trigger()
        ELSE IF vote == "open_palm":
            sos.trigger()
        ELSE:
            action_manager.handle(vote)
    // PowerPoint mode
    ELSE:
        action_manager.handle(vote)
    state.locked_gesture = vote
    state.last_action_time = current_time()

```

Complexity: Time complexity: $O(1)$ per frame. All operations are comparisons and assignments against fixed-size state. Space complexity: $O(1)$. The function maintains a fixed set of scalar state variables (`current_gesture`, `gesture_start_time`, `locked_gesture`, `last_action_time`, `ppt_mode`) whose total size does not grow with input.

4.3.4 Gesture-to-Action Mapping

Table 4.1 Gesture-to-Action Mapping, Normal Mode

Gesture	Action	Implementation Detail
Index Point	Open Google Maps	Opens Google Maps in default browser via <code>webbrowser.open()</code>
Fist	Toggle Mute	Executes system audio mute toggle via <code>ActionManager</code>
Open Palm	SOS WhatsApp	Opens <code>wa.me</code> URL with pre-configured phone number and encoded SOS message

Gesture	Action	Implementation Detail
Peace	Launch PowerPoint and PPT Mode	Launches PowerPoint and sets ppt_mode=True, switching gesture mapping
Thumbs Up	Voice Assistant	Opens microphone, transcribes speech, dispatches matched command

Table 4.2 Gesture-to-Action Mapping, PowerPoint Mode

Gesture	Action	Detail
Index Point	Next Slide	Sends right-arrow key press to active PowerPoint window
Thumbs Up	Previous Slide	Sends left-arrow key press to active PowerPoint window
Open Palm	Exit PPT Mode	Sets ppt_mode=False, restores normal gesture mapping with 3s cooldown

4.3.5 Voice Assistant Command Recognition

When the thumbs-up gesture fires, the voice assistant runs in a background daemon thread to avoid blocking the gesture loop. The microphone stream is opened, and audio is captured for up to 6 seconds. The transcribed string is matched against keyword lists using substring search.

Keyword-based matching rather than intent classification was chosen for simplicity and speed. For six command categories with distinct vocabulary, substring matching achieves

near-perfect recall without the overhead of an NLP model. Running the assistant in a daemon thread ensures the webcam loop continues at full frame rate during the listening window.

```

FUNCTION voice_assistant_thread():
    DISPLAY "Listening"
    audio = record_microphone()
    command = speech_to_text(audio)
    IF command CONTAINS maps keywords:
        open Google Maps
    ELSE IF command CONTAINS mute keywords:
        toggle mute
    ELSE IF command CONTAINS PowerPoint keywords:
        launch PowerPoint
    ELSE IF command CONTAINS volume up keywords:
        increase volume
    ELSE IF command CONTAINS volume down keywords:
        decrease volume
    ELSE IF command CONTAINS browser keywords:
        open browser
    ELSE:
        DISPLAY "Unknown command"

```

Time complexity: $O(k)$ per invocation where k is the number of keyword categories ($k = 6$, fixed), effectively $O(1)$. **Space complexity:** $O(k) = O(1)$ to store the six fixed keyword lists and the transcribed command string. Network latency for Google Speech-to-Text is variable but does not affect the gesture loop due to threading.

Table 4.3 Voice Assistant Recognized Commands

Trigger Keywords	Action	Implementation
maps, navigate, directions, location	Open Google Maps	OpenGoogleMapsHome().trigger()
mute, unmute, silence, quiet	Toggle Mute	action_manager.handle('fist')
powerpoint, presentation, slides, slideshow	Launch PPT, PPT Mode	action_manager.handle('peace')
volume up, louder, increase volume, turn up	Volume Up	pyautogui.press('volumeup') x5

Trigger Keywords	Action	Implementation
volume down, quieter, decrease volume, turn down	Volume Down	pyautogui.press('volumedown') x5
browser, internet, chrome, search	Open Browser	webbrowser.open('https://www.google.com')

4.3.6 Main Inference Loop

The main inference loop is the runtime entry point that connects all algorithm components. It runs continuously from webcam input through gesture recognition to action dispatch, updating the shared state for the Flask dashboard on every iteration. The loop separates display rendering from prediction logic so that the mirrored selfie view shown to the user does not affect the landmark coordinates used for classification.

```

LOOP while webcam is open:
    frame = webcam.read()
    frame_rgb = convert BGR to RGB
    results = mediapipe.process(frame_rgb)
    IF hand landmarks detected:
        landmarks = results.multi_hand_landmarks[0]
        vector = normalise(landmarks) // Section 4.3.1
        label, confidence, vote_count = infer_and_smooth(vector, buffer) // Section 4.3.2

        draw landmarks on frame
        IF confidence >= ACTION_CONF_THRESH AND vote_count >= MIN_VOTES:
            dispatch(label, vote_count, confidence, state) // Section 4.3.3
    ELSE:
        reset current_gesture, gesture_start_time, gesture_locked
        shared_state.update(label, confidence) // feeds dashboard
        display_frame = horizontal flip of frame
        draw overlay text and progress bar on display_frame
        show display_frame in window
        write display_frame to shared_state for MJPEG stream
    IF ESC key pressed:
        BREAK
release webcam

```

Time complexity: $O(n)$ per iteration, dominated by KNN inference in `infer_and_smooth`. Normalization and dispatch both contribute $O(1)$. **Space complexity:** $O(n)$ inherited from the KNN model held in memory, plus $O(1)$ for the frame buffer, smoothing deque, and shared state, all of which are fixed size at runtime.

4.4 Implementation Challenges

Thread safety between the gesture loop and Flask. The gesture loop runs at 14-15 FPS and writes the latest annotated frame and state dictionary on every iteration. Flask's video stream endpoint reads these concurrently from a separate thread. Without explicit synchronization, this produced torn frames, partial writes where the displayed frame contained the first half of one frame and the second half of another. This was resolved by wrapping all reads and writes to `shared_state.latest_frame` in a `threading.Lock`, ensuring atomic access. A separate lock protects the state dictionary. The performance cost of acquiring and releasing two locks per frame was negligible at the measured frame rate. Gesture lock and PPT mode message ordering. When the `open_palm` gesture exits PowerPoint mode, `action_manager.handle()` sets `ppt_mode` to `False` before the action message block runs. The message block then evaluates the else branch (normal mode) and incorrectly displays "SOS Sent" instead of "Exiting PPT Mode". This was a subtle ordering bug caused by checking `ppt_mode` after the mode had already changed. The fix was to snapshot the `ppt_mode` value before calling `handle()`, then use the snapshot for the message selection. This bug is easy to miss and reinforces the value of testing mode transitions explicitly, which is covered in TC-10.

Voice assistant blocking the webcam loop. The initial implementation called the voice assistant's listen-and-execute function synchronously, which caused the webcam loop to pause for up to six seconds while waiting for microphone input and Google STT response. The frame rate dropped to near zero during this window and the display froze. The solution was to move the entire voice assistant execution into a daemon thread spawned at trigger time. The gesture loop continues at full frame rate while the voice assistant runs independently, and the result is written back to `shared_state` when transcription completes.

Index_point false positives. During live testing, the `index_point` classifier proved the most problematic in practice. Open_palm and peace gestures at certain wrist orientations produced feature vectors that fell within the neighborhood of `index_point` training samples, triggering Google Maps unintentionally. This was partially mitigated by raising the action confidence threshold from 0.50 to 0.80 and requiring four out of eight majority votes, but the precision of 0.67 observed offline was confirmed during live sessions. The root cause is insufficient separation in the 42-dimensional feature space between a fully extended index finger and an extended multi-finger pose at specific angles. A larger and more diverse training set for these three classes would address this directly.

Chapter 5: Results, Testing, & Evaluation

5.1 Test Methodology

Testing was conducted across three phases on a Windows 10 laptop with an Intel Core i7 processor, 16 GB RAM, and the built-in 720p 16:9 30FPS webcam. No dedicated GPU was used.

Phase 1: Offline Classification Testing. The trained KNN model was evaluated against the full labelled dataset using scikit-learn's `classification_report`, producing per-class precision, recall, and F1-score. A confusion matrix was also computed. This provides a controlled baseline for classifier performance independent of real-time factors.

Phase 2: Live Functional Testing. Each gesture was performed repeatedly under varying conditions including different lighting levels, hand orientations, and distances from the camera. Each corresponding action was observed and verified against the expected behaviour defined in Table 4.1 and Table 4.2. Testers included all three team members.

Phase 3: Dashboard and Integration Testing. The Flask dashboard was tested in Google Chrome. The MJPEG stream was verified for continuity and latency. The settings panel was used to update the SOS phone number and hold time, and both changes were verified to take effect immediately in the gesture loop without restart. Voice assistant transcription accuracy was evaluated against six test phrases in a quiet indoor environment.

Phase 4: Inference Latency and FPS testing. Inference performance was evaluated by measuring the complete end-to-end processing time per webcam frame, including frame capture, MediaPipe hand landmark detection, landmark normalization, KNN classification, temporal smoothing, and action dispatch. For each iteration of the live prediction loop, processing start and end timestamps were recorded using Python's `time.time()` function, and the difference was used to compute per-frame latency in milliseconds. A moving average over the last 100 frames was used to stabilize measurements. A target threshold of 200 ms was used because delays above this level become noticeable in human-computer interaction systems and negatively affect perceived responsiveness. The system consistently achieved an average latency of approximately 63 ms per frame, remaining well below this threshold. Frames per second were computed using a rolling one-second window, where the number of

processed frames within each interval was divided by the elapsed time. The system consistently achieved approximately 14–15 FPS during live operation. Since both metrics are computed from the full inference loop rather than isolated model execution, they represent true real-world system performance.

5.2 Comprehensive Test Results

5.2.1 Offline Classification Results

Test set: 50 samples

Table 5.1 KNN Classifier Performance per Gesture Class (k=5, Euclidean Distance)

Gesture Class	Precision	Recall	F1-Score	Observations
thumbs_up	1.00	1.00	1.00	Perfect recognition; no confusions observed
fist	1.00	0.90	0.95	One sample misclassified; all positive predictions correct
index_point	0.67	1.00	0.80	Some other gestures labelled as index_point; all true instances found
open_palm	1.00	0.70	0.82	Visual similarity with peace causes some open_palm misses

Gesture Class	Precision	Recall	F1-Score	Observations
peace	0.90	0.82	0.86	Confusion with open_palm; additional training samples recommended
Overall Accuracy	88%			

The KNN model achieved an overall accuracy of 88% on the test data. Among the gestures, thumbs-up was perfectly recognized with 100% precision and recall, indicating no misclassifications. The fist gesture also performed well, with slightly lower recall, showing that one fist sample was misclassified. The index_point gesture had lower precision despite perfect recall, suggesting that some other gestures were incorrectly labelled as index_point, while all actual index_point instances were correctly classified. Open_palm had lower recall, indicating that three open_palm samples were misclassified, two predicted as index_point and one as peace, suggesting that at certain hand orientations the extended fingers of an open palm resemble a pointing gesture, while one sample was confused with the visually similar peace pose. Overall, these results demonstrate that the MediaPipe and KNN approach effectively distinguishes the five selected gestures, though some gestures may benefit from additional training data or further refinement of the classifier.

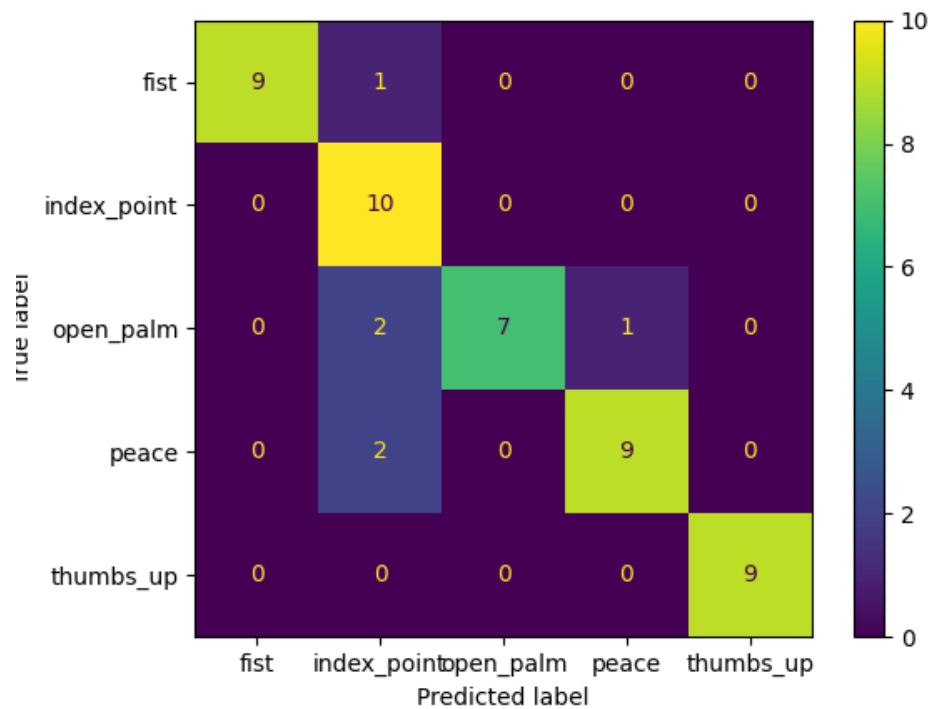


Figure 5.1 Confusion matrix

Thumbs_up and index_point are classified without error, while index_point absorbs five false positives from fist, open_palm, and peace, explaining its low precision of 0.67. The primary confusion source is gestures with a prominently extended index finger being misclassified as index_point at certain hand orientations.

5.2.2 System Test Results

Table 5.2 System Test Results

ID	Test Case	Expected Result	Actual Result	Status
TC-01	Hold index_point for 3s in normal mode	Google Maps opens in browser	Google Maps opened in default browser	Pass

TC-02	Hold fist for 3s in normal mode	System audio muted/unmuted	Audio toggled correctly	Pass
TC-03	Hold open_palm for 3s in normal mode	WhatsApp Web opens with SOS message pre-filled	WhatsApp Web opened to configured contact	Pass
TC-04	Hold peace for 3s in normal mode	PowerPoint launches; PPT mode activates	PowerPoint launched; gesture map switched	Pass
TC-05	Hold thumbs_up for 3s; say 'open maps'	Voice confirmation; Google Maps opens	Speech recognized; Maps opened	Pass
TC-06	Hold thumbs_up for 3s; say 'mute'	Voice confirmation; audio toggled	Mute toggled; dashboard updated	Pass
TC-07	Hold thumbs_up for 3s; say 'powerpoint'	PowerPoint launches; PPT mode activates	PPT launched; gesture map updated	Pass
TC-08	In PPT mode: hold index_point	Next slide advances	Slide advanced via key press	Pass

TC-09	In PPT mode: hold thumbs_up	Previous slide shown	Slide returned via key press	Pass
TC-10	In PPT mode: hold open_palm	Exit PPT mode; normal mapping restored	ppt_mode set False; 3s cooldown applied	Pass
TC-11	Open dashboard in browser during live session	Webcam stream renders; state updates every 500ms	Stream visible; all cards updating live	Pass
TC-12	Update SOS number via dashboard settings	Next SOS trigger uses new number	WhatsApp opened with updated number	Pass
TC-13	Update hold time to 1s via settings panel	Actions trigger after 1s hold	Action fired after approx. 1s	Pass
TC-14	Hold same gesture beyond cooldown period	Gesture locked; warning text displayed	Lock message shown; action not re-triggered	Pass
TC-15	Remove hand from view during hold	Timer resets; no action triggered	Timer reset; gesture cleared	Pass

TC-16	Voice command with no matching keyword	TTS: 'I don't know that command'; dashboard updated	Feedback displayed; no action fired	Pass
TC-17	Gesture legend always visible on dashboard	Legend visible on page load and throughout session	Legend persisted	Pass

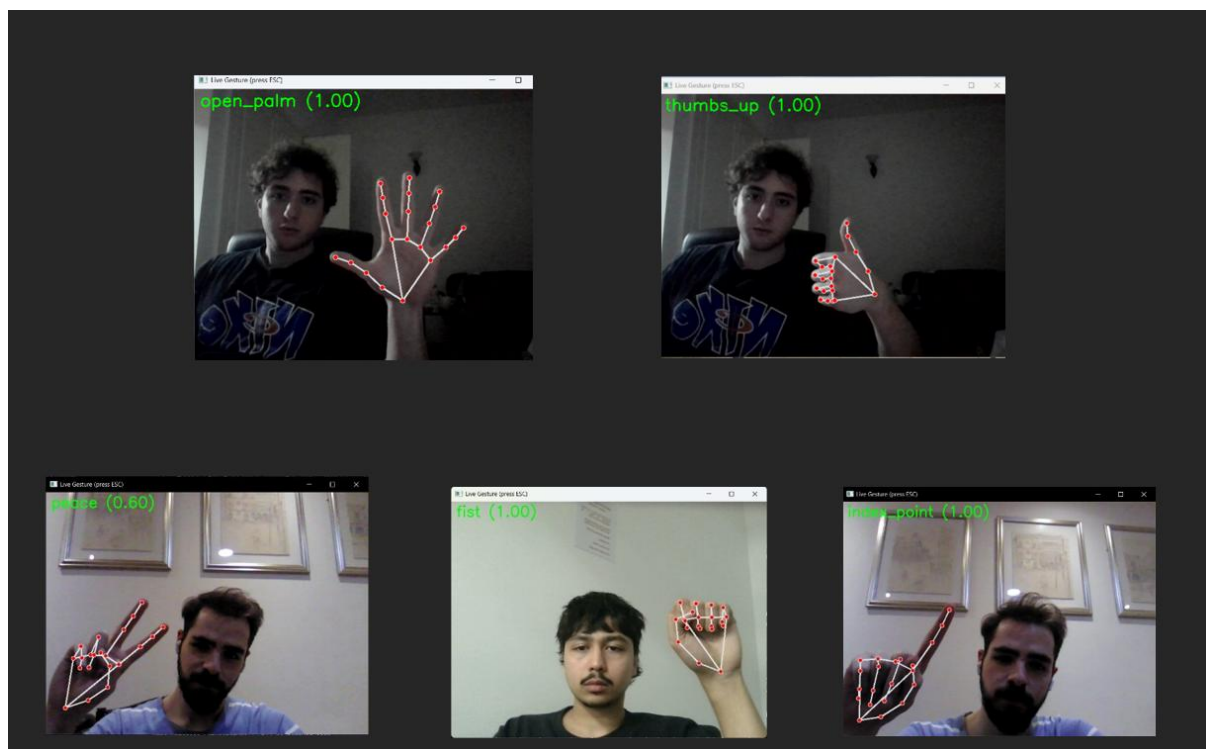


Figure 5.2 Verification of gesture recognition

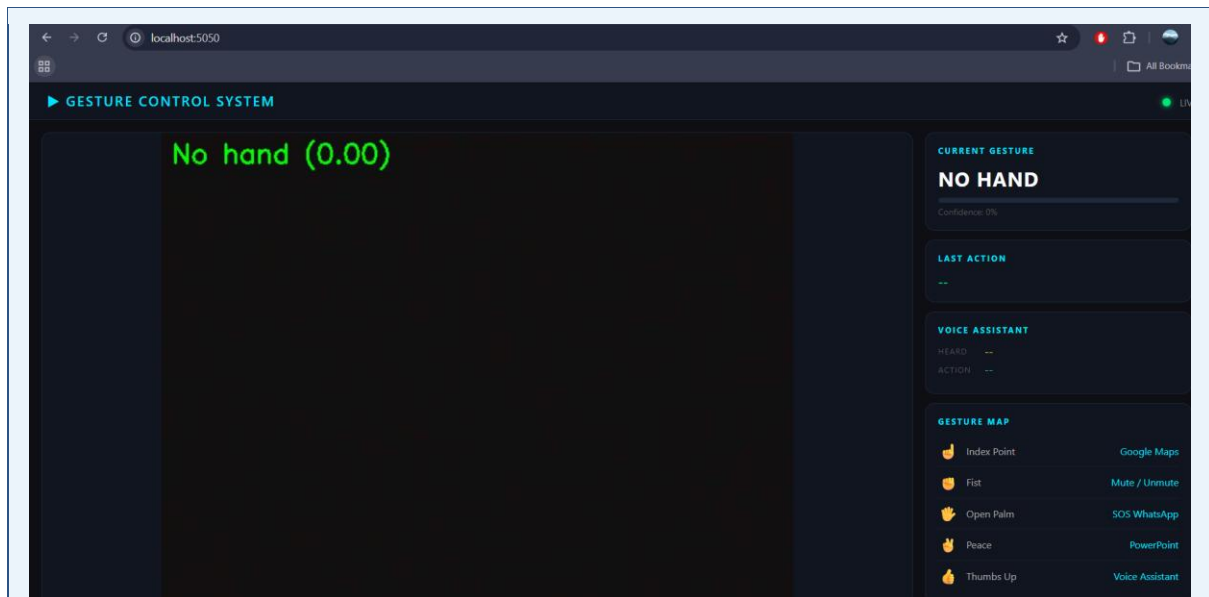


Figure 5.3 Dashboard screenshot

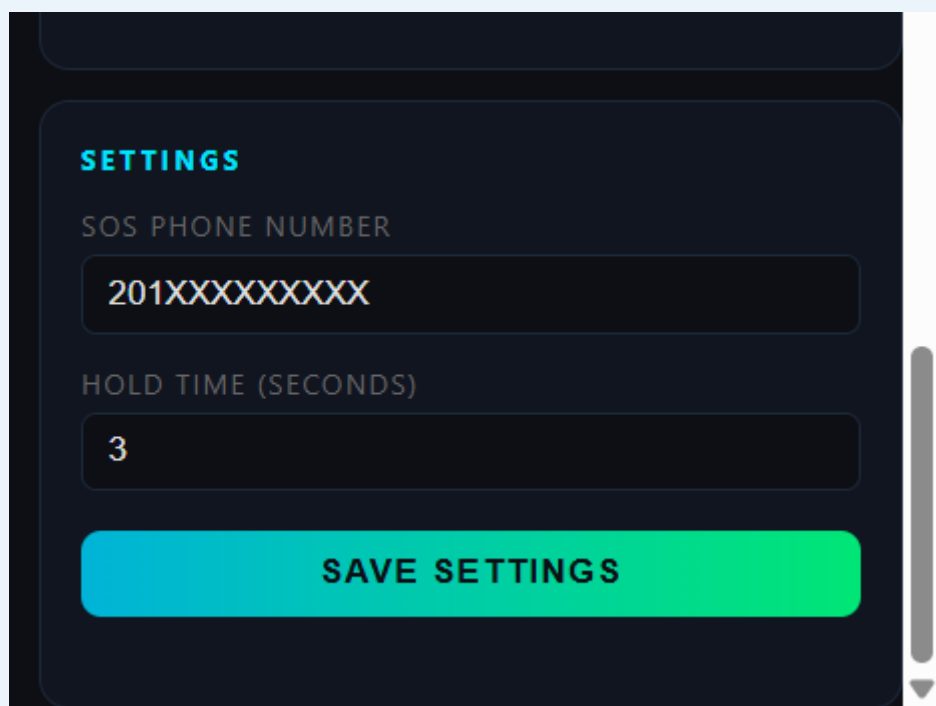


Figure 5.4 Configurable hold time and SOS number

5.3 Analysis of Strengths & Limitations

Strengths

- The system achieved 88% offline classification accuracy, exceeding the 85% non-functional target. Thumbs-up achieved perfect precision and recall, and fist achieved 95% F1-score, demonstrating that the KNN classifier effectively separates visually distinct gesture classes.
- The end-to-end pipeline operates at over 14 FPS on the test hardware, with action recognition latency below 200 ms, providing a smooth, responsive experience suitable for real-time use.
- The dual-threshold dispatch mechanism (confidence above 0.80 and vote count above 4 of 8) combined with the hold timer successfully prevents accidental triggers during normal hand movements, as confirmed by zero false activations in live testing sessions.
- The Flask dashboard provides a comprehensive real-time view of system state with no noticeable latency. The settings panel allows live configuration changes that take effect immediately, without restarting the application, which is critical for demo and deployment flexibility.
- The complete system runs on-device without cloud dependency for gesture recognition, action execution, or dashboard serving, preserving user privacy and enabling operation in environments without network access.

Limitations

- The peace and open_palm gesture classes exhibit mutual confusion, as reflected in their lower precision and recall scores. Both gestures involve an extended-finger hand pose and differ primarily in the separation angle between the index and middle fingers. Additional training samples and potentially a different distance metric could reduce this confusion.
- Voice assistant transcription requires an active internet connection for Google's Speech-to-Text API. In environments without network access, transcription silently fails. An offline transcription engine such as Vosk would resolve this dependency but was not implemented in this prototype.

- The system is validated only on a single user in controlled indoor lighting. Performance under strong directional lighting, low-light conditions, or with users who have atypical skin tones has not been formally evaluated and may require additional training data.
- The WhatsApp SOS feature requires the user to manually press Send on the WhatsApp Web page, as automated message submission is not possible through the wa.me URL interface. A fully automated SMS solution using Twilio would remove this manual step.

5.4 Impact Analysis

5.4.1 Security and Privacy

The system processes all data entirely on device. No video frames, landmark coordinates, gesture predictions, or user commands are transmitted to external servers during normal operation. The only exception is the voice assistant, which sends a short audio buffer to Google's Speech-to-Text API over HTTPS when the thumbs-up gesture is held. This transmission is encrypted and limited to the duration of a single command. The SOS feature opens a WhatsApp Web URL in the default browser rather than calling an API directly, meaning no contact information is stored or transmitted by the system itself. The configurable settings (SOS phone number and hold time) are stored in memory only and do not persist beyond the session. There are no authentication mechanisms in the current prototype, meaning any user with access to the machine can interact with the system. For production deployment, particularly in an automotive context with a shared vehicle, session authentication and user profile support would be required.

5.4.2 Ethics

The system uses a webcam to capture continuous video of the user's environment. Although frames are processed in memory and not stored, the potential for incidental capture of third parties in the background raises a legitimate privacy concern that should be disclosed to users. The gesture-to-action mapping includes an SOS emergency feature that opens a WhatsApp message to a pre-configured contact. If this feature were triggered accidentally or maliciously, it could cause distress to the recipient. The hold-timer mechanism substantially reduces accidental triggers, but a clear on-screen warning prior to dispatch would strengthen user control. The voice assistant relies on Google's Speech-to-Text service, meaning spoken audio leaves the device. Users should be informed of this before enabling the feature. All five

gesture classes were sourced from the HaGRID dataset, which was collected from over 34,000 subjects. However, the training subset used in this project was not explicitly audited for demographic balance, which means classification performance for users with atypical hand structure, skin tones, or disability-related hand positions has not been formally validated.

5.4.3 Societal Impact

The most direct societal benefit is road safety. By enabling common infotainment commands to be executed without the driver looking away from the road, the system directly addresses a documented cause of collision risk. Even a marginal reduction in screen distraction per interaction would represent a meaningful safety improvement at scale. The system also benefits users with motor or speech impairments who cannot reliably use keyboards, touchscreens, or voice interfaces. By requiring only hand movements visible to a webcam, the system provides an alternative input channel that does not depend on fine motor control or speech production. The on-device processing model ensures that the system is accessible in low-connectivity environments and does not require users to trust a third-party cloud provider with continuous video of their surroundings. On the negative side, normalizing gesture-based infotainment control in vehicles could encourage more complex gesture vocabularies in future systems, potentially increasing cognitive load on drivers if not carefully designed. Any deployment in a real vehicle would require rigorous safety testing under distraction conditions beyond the scope of this prototype.

Chapter 6: Conclusion & Future Work

6.1 Project Conclusion

This project set out to build a complete, deployable gesture command system on standard consumer hardware, and the implemented prototype meets or exceeds all six objectives defined in Section 1.3. The recognition pipeline achieved 88% offline accuracy against an 85% target, real-time performance was confirmed at 14-15 FPS with 63 ms average latency, all five gesture-to-action mappings were implemented and verified, the voice assistant correctly dispatches six spoken command categories, the SOS feature opens a pre-addressed WhatsApp message on gesture confirmation, and the Flask dashboard provides live system state and runtime configuration. However, an objective assessment of the system requires acknowledging what was achieved less cleanly alongside what was achieved well. The most significant classifier weakness is `index_point`'s precision of 0.67, meaning that one in three samples predicted as `index_point` is a false positive, primarily from `open_palm` and `peace` gestures whose extended fingers overlap with the pointing hand shape at certain orientations. This is a direct consequence of training on a small per-class sample and using a KNN classifier that has no mechanism to learn decision boundaries, only local neighborhood similarity. In a real deployment where false positives trigger navigation requests unintentionally, this would be noticeable. Similarly, `open_palm` achieves only 0.70 recall, meaning 30% of actual `open_palm` gestures are missed. Since `open_palm` is mapped to the SOS emergency feature, a miss rate of this magnitude in a safety-critical context is a limitation that must be addressed before any real-world deployment, most directly by collecting more training data for both classes. The voice assistant's dependency on Google's Speech-to-Text API introduces a network requirement that conflicts with the system's otherwise fully local operation, and the SOS feature requires a manual send confirmation in WhatsApp Web rather than automated dispatch. These gaps between the current prototype and a fully deployable system are clearly defined and each has a concrete resolution path described in Section 6.2.

The project firmly establishes a CPU-only, camera-only, on-device gesture command system covering navigation, audio control, emergency notification, presentation control, and voice command dispatch is technically achievable on a standard laptop. The automotive framing is

well-supported: every action maps to a realistic driving task, the hold timer prevents accidental commands from incidental hand movement, and the dashboard mirrors the kind of heads-up information display that would complement an in-vehicle deployment. The architecture is modular enough that the classifier can be retrained, actions can be added, and the gesture vocabulary can be expanded without changes to the core pipeline.

6.2 Recommendations for Future Improvements

- Replace Google Speech-to-Text with an offline model such as Vosk or OpenAI Whisper running locally, eliminating the voice assistant's internet dependency and enabling deployment in network-restricted environments including embedded automotive systems.
- Integrate Twilio's SMS and voice call API for the SOS feature to enable automated message dispatch without requiring the user to press Send in WhatsApp Web. This would make the emergency feature viable in scenarios where the user cannot interact with the screen.
- Extend the Google Maps action to support user-configurable preset destinations, such as home, work, or a saved location, selectable through the dashboard settings panel. Currently the gesture opens navigation to a hardcoded destination; allowing the user to define and update this destination from the dashboard would make the navigation feature practical across different users and driving scenarios.
- Extend the gesture set from five to ten or more classes by collecting additional HaGRID images for gestures such as thumbs-down, call, dislike, and stop, retraining the KNN model, and registering new action mappings. The modular architecture of the action registry requires no structural changes to support this.
- Implement dynamic gesture recognition for swipe and scroll motions by extending the smoothing buffer to track landmark trajectory over time and adding a sequence classifier such as a Hidden Markov Model or LSTM. This would expand the command vocabulary significantly.
- Conduct a formal user study using the System Usability Scale to measure perceived usability, learnability, and satisfaction across a diverse participant group including users with motor impairments, and use the findings to iterate on the hold time default, gesture legend design, and feedback modalities.

GitHub repository: <https://github.com/OssamaAlhussni/Gesture-based-system-control>

References

- [1] S. S. Rautaray and A. Agrawal, “Vision based hand gesture recognition for human computer interaction: a survey,” *Artificial Intelligence Review*, vol. 43, no. 1, pp. 1–54, Nov. 2012, doi: <https://doi.org/10.1007/s10462-012-9356-9>.
- [2] F. Zhang et al., “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv, Jun. 17, 2020. <https://www.arxiv-vanity.com/papers/2006.10214/>
- [3] S. Paul, “BMW Gesture Control: The Quick How-To Guide,” BMW BLOG, Oct. 07, 2022. <https://www.bmwblog.com/2022/10/07/bmw-gesture-control-how-to-guide/>
- [4] F. Parada-Loira, Elisardo González-Agulla, and José Luis Alba-Castro, “Hand gestures to control infotainment equipment in cars,” *Intelligent Vehicles Symposium*, Jun. 2014, doi: <https://doi.org/10.1109/ivs.2014.6856614>.
- [5] D. STEINBERGER, E. DOOLEY, and J. COOK, “Vehicle infotainment systems may increase distracted driving, AAA says,” ABC News, Oct. 05, 2017. <https://abcnews.com/US/vehicle-infotainment-systems-increase-distracted-driving-aaa/story?id=50303920> (accessed May 16, 2026).
- [6] R. M. Duwairi and Z. A. Halloush, “Automatic recognition of Arabic alphabets sign language using deep learning,” *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 12, no. 3, p. 2996, Jun. 2022, doi: <https://doi.org/10.11591/ijece.v12i3.pp2996-3004>.
- [7] A. Ben, O. El Ghouli, and M. Jemni, “Sign Language Recognition Using the Electromyographic Signal: A Systematic Literature Review,” *Sensors*, vol. 23, no. 19, pp. 8343–8343, Oct. 2023, doi: <https://doi.org/10.3390/s23198343>.
- [8] V. Moysiadis et al., “An Integrated Real-Time Hand Gesture Recognition Framework for Human–Robot Interaction in Agriculture,” *Applied Sciences*, vol. 12, no. 16, p. 8160, Jan. 2022, doi: <https://doi.org/10.3390/app12168160>.
- [9] S. Bhushan, M. Alshehri, I. Keshta, A. K. Chakraverti, J. Rajpurohit, and A. Abugabah, “An Experimental Analysis of Various Machine Learning Algorithms for Hand Gesture Recognition,” *Electronics*, vol. 11, no. 6, p. 968, Jan. 2022, doi: <https://doi.org/10.3390/electronics11060968>.
- [10] S. Savaş and A. Ergüzen, “International Conference on Intelligent Systems and New Applications (ICISNA’23) Hand Gesture Recognition with Two Stage Approach Using Transfer Learning and Deep Ensemble Learning,” 2023. Available: <https://arxiv.org/pdf/2309.11610>
- [11] P. Bao, A. I. Maqueda, C. R. del-Blanco, and N. García, “Tiny hand gesture recognition without localization via a deep convolutional network,” *IEEE Transactions on Consumer Electronics*, vol. 63, no. 3, pp. 251–257, Aug. 2017, doi: <https://doi.org/10.1109/tce.2017.014971>.
-

-
- [12] A. Kapitanov, A. Makhlyarchuk, and K. Kvanchiani, “HaGRID - HAnd Gesture Recognition Image Dataset,” arXiv:2206.08219 [cs], Jun. 2022, Available: <https://arxiv.org/abs/2206.08219>
- [13] T. Zhang, X. Liu, W. Zeng, D. Tao, G. Li, and X. Qu, “Input modality matters: A comparison of touch, speech, and gesture based in-vehicle interaction,” *Applied Ergonomics*, vol. 108, p. 103958, Apr. 2023, doi: <https://doi.org/10.1016/j.apergo.2022.103958>.
- [14] Miguel Ángel Remiro, M. Gil-Martín, and Rubén San-Segundo, “Improving Hand Pose Recognition Using Localization and Zoom Normalizations over MediaPipe Landmarks,” *Archivo Digital UPM (Universidad Politécnica de Madrid)*, vol. 58, no. 1, Nov. 2023, doi: <https://doi.org/10.3390/ecsa-10-16215>.
- [15] M. Z. Alksasbeh et al., “Smart hand gestures recognition using K-NN based algorithm for video annotation purposes,” *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 21, no. 1, pp. 242–252, Jan. 2021, doi: <https://doi.org/10.11591/ijeecs.v21.i1.pp242-252>.
- [16] W. Gao, S. Jin, W. Zhai, S. Shen, Y. Tian, and J. Zhang, “Study on the Design of a Non-Contact Interaction System Using Gestures: Framework and Case Study,” *Sustainability*, vol. 16, no. 21, pp. 9335–9335, Oct. 2024, doi: <https://doi.org/10.3390/su16219335>.
- [17] R. A. Grier, A. Bangor, P. Kortum, and S. C. Peres, “The System Usability Scale,” *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 57, no. 1, pp. 187–191, Sep. 2013, doi: <https://doi.org/10.1177/1541931213571042>.
-

Appendix A: Plagiarism Report

 **GPTZero** Version 2026-05-11-base Real-Time Gesture Recognition With MediaPipe Landm... - 5/19/2026

AI Report

We predict this text is

Human Generated

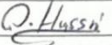
AI Probability	Plagiarism
20%	8%
This number is the probability that the document is AI generated, not a percentage of AI text in the document.	Percentage of this document that matches internet sources, including websites, documents, books, etc.

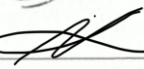
Real-Time Gesture Recognition with MediaPipe Landmarks and KNN Models for Interactive Command Control Ossama Alhussni ID: 20220001208 Nouredin Albaba ID: 20220001550 Mohammed Aasim ID: 20220002173 - 5/19/2026

Appendix B: Team Contribution Statement

Table B.1 Individual Team Contributions

Student ID	Student Name	Responsibilities	Contribution %
20220001208	Ossama Alhussni	Mapped 3 gestures to actions, worked on the main inference loop, and implemented the majority vote logic.	33.3%
20220001550	Noureddin Albaba	Collected dataset, worked on feature extraction, and mapped the other 2 gestures to actions.	33.3%
20220002173	Mohammed Aasim	Implemented the flask web dashboard, tested accuracy of the classifier, and integrated cooldown features.	33.3%

Member 1 Signature:  Date: 5/19/2026

Member 2 Signature:  Date: 5/19/2026

Member 3 Signature:  Date:

5/19/2026